Physics-Informed Neural Networks (PINNs): General Concept

DIADEM Summer School 2026

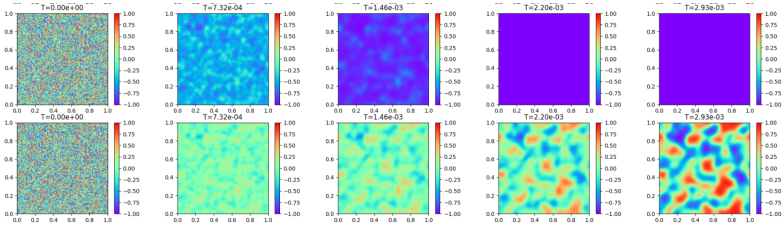
Chih-Kang Huang

Centre Inria d'Université Côte d'Azur & Laboratoire J.A. Dieudonné

September 3rd 2026

Limitations of Data-Driven ML in Materials Science

- **Data-Hungry:** High-fidelity DFT and experiments are extremely limited; standard networks starve for data.
- **Prone to Generalization Failure:** Black-box models fail outside their training domain.



Prediction of a heterogeneous mixture of two phases by Neural Nets

- **Unphysical Behavior:** Pure regression violates conservation laws, thermodynamic bounds, and sharp interface scales (ϵ).

Why not use **physical laws (PDEs)** *to guide neural network training?*

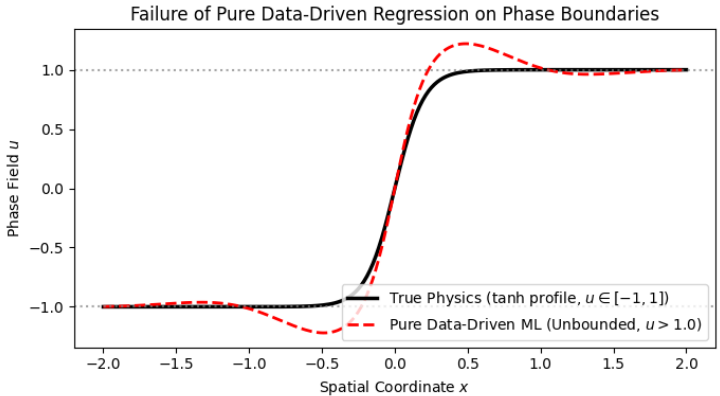
The Failure of Pure Regression: Phase Separation

Alloy solidification:

- State variable $u(t, x) \in [-1, 1]$ tracks phase separation.
- Governed by Allen-Cahn: $\partial_t u = \epsilon^2 \Delta u - (u^3 - u)$.

The Failure & Fix:

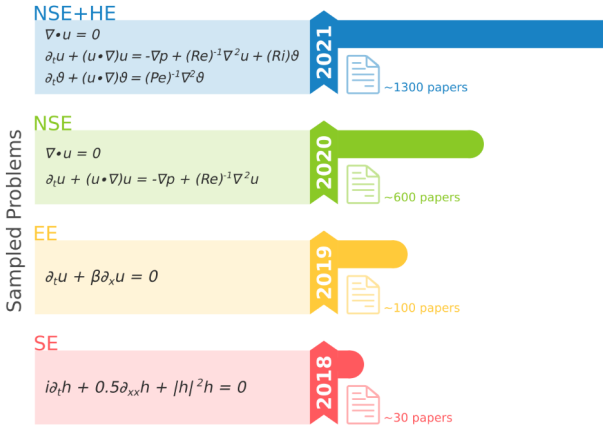
- Pure ML could predict unphysical bounds ($u > 1$).
- **Idea:** Add PDE residuals to the training loss $\rightarrow$ PINNs.



Outline

A Short History of PINNs

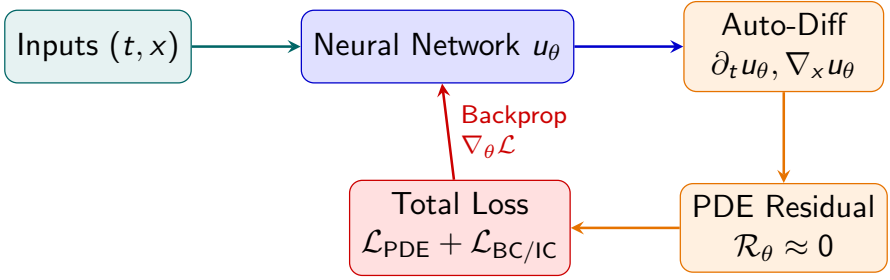
- **1990s (Early Pioneers):** Neural networks for simple PDEs (Dissanayake & Phan-Thien 1994, Lagaris et al. 1998). Limited by hardware and optimization.
- **2017–2019 (Modern Formalization):** Modern PINNs framework via automatic differentiation (Raissi, Perdikaris, & Karniakadis 2017/2019).
- **Today:** Explosive adoption across physics, engineering, and materials science.



Literature growth (Cuomo et al.)

Key Idea:

- Approximate the solution by a neural network surrogate $u_\theta(t, x)$.
- Embed governing PDEs and boundary conditions as a **soft constraint** in the loss function.
- **Unsupervised Learning:** No simulation data required for training.



Mathematical Formulation & Mechanics

To solve $\partial_t u = \mathcal{D}[u]$, we minimize the weighted composite loss:

$$\mathcal{L}_{phys}(\theta) = \lambda_{PDE}\,\mathcal{L}_{PDE} + \lambda_{IC}\,\mathcal{L}_{IC} + \lambda_{BC}\,\mathcal{L}_{BC}, \quad \text{(loss weights } \lambda > 0\text{)}$$

where residuals are evaluated at *collocation points* (t_i, x_i), e.g., the PDE residual:

$$\mathcal{L}_{PDE}(\theta) = \frac{1}{N}\sum_{i=1}^{N} |\partial_t u_\theta(t_i, x_i) - \mathcal{D}[u_\theta(t_i, x_i)]|^2$$

Implementation Mechanics

- **Derivatives:** Primarily Automatic Differentiation (JAX, PyTorch); classical schemes (finite differences, FFT) are also applicable.
- **Mesh-Free:** Domain sampled randomly or adaptively without mesh generation.
- **Activation Function:** Use smooth activations (tanh, GELU, etc.). **Never use ReLU** ($\partial_{xx}\text{ReLU} \equiv 0$).

Example: 1D Heat Equation

Problem Setup: $\partial_t u = u_{xx}$ for $(t, x) \in (0, T] \times (-1, 1)$ with $u(0, x) = h(x)$ and $u(t, \pm 1) = 0$.

Loss Decomposition:

$$\mathcal{L}_{PDE} = \frac{1}{N_{pde}} \sum_{i=1}^{N_{pde}} \left(\partial_t u_\theta(t_i, x_i) - \partial_{xx} u_\theta(t_i, x_i)\right)^2$$

$$\mathcal{L}_{IC} = \frac{1}{N_{ic}} \sum_{j=1}^{N_{ic}} \left(u_\theta(0, x_j) - h(x_j)\right)^2$$

$$\mathcal{L}_{BC} = \frac{1}{N_{bc}} \sum_{k=1}^{N_{bc}} \left|u_\theta(t_k, \pm 1)\right|^2$$

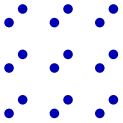
As $\mathcal{L}_{phys}(\theta) \to 0 \implies u_\theta(t, x)$ *converges to the exact PDE solution.*

Hands-on implementation (Heat & Allen–Cahn equations)

Overcoming the Curse of Dimensionality

Classical Solvers (FEM/FDM):

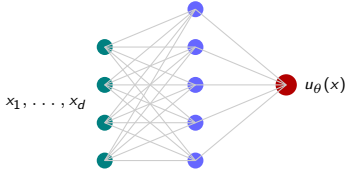
- Rely on spatial grids/meshes.
- Complexity scales as $\mathcal{O}(N^d) \rightarrow$ Curse of Dimensionality (CoD).
- Infeasible for $d \geq 4$.



Mesh points: N^d (exponential)

PINNs (Mesh-Free):

- Approximates $u(x, t)$ continuously.
- Complexity scales with NN width/depth, not grid size.
- Fast inference once trained.



Theoretical Result (de Ryck et al., 2022)

For non-linear parabolic PDEs (e.g., Allen–Cahn, Heat equation), the required NN size grows only **polynomially** in dimension: $\mathcal{O}(\text{poly}(d))$.

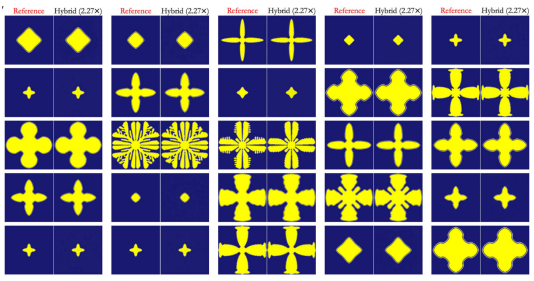
Theory vs. Practice: Hybrid Frameworks

The Practical Bottleneck:

- While PINNs theoretically break CoD, non-convex optimization in high dimensions remains notoriously difficult.

Alternatives: Hybrid Solvers

- Combine classical numerical steps (guaranteeing precision) with NN predictions (providing acceleration).
- *Example:* Alternating solver-NN iterations (Oommen et al., 2024).



Hybrid coupling of classical solvers and NNs (Oommen et al., 2024)

Outline

Advanced Techniques for PINN Convergence

1. **Adaptive Sampling (R3, RAR)**
   - **Solves:** Localized shocks & steep gradients.
   - Concentrates points where residual $|\mathcal{R}_\theta(x)|$ is high.

Advanced Techniques for PINN Convergence

1. **Adaptive Sampling (R3, RAR)**
   - **Solves:** Localized shocks & steep gradients.
   - Concentrates points where residual $|\mathcal{R}_\theta(x)|$ is high.
2. **Adaptive Weighting**
   - **Solves** Gradient imbalance across loss terms.
   - Rescales weights so IC/BCs aren't dominated by PDE.

Advanced Techniques for PINN Convergence

1. **Adaptive Sampling (R3, RAR)**
   - **Solves:** Localized shocks & steep gradients.
   - Concentrates points where residual $|\mathcal{R}_\theta(x)|$ is high.
2. **Adaptive Weighting**
   - **Solves** Gradient imbalance across loss terms.
   - Rescales weights so IC/BCs aren't dominated by PDE.
3. **Causality Training**
   - **Solves** Temporal instability & unphysical states.
   - Enforces sequential learning from $t = 0 \to T$.

Advanced Techniques for PINN Convergence

1. **Adaptive Sampling (R3, RAR)**
   - **Solves:** Localized shocks & steep gradients.
   - Concentrates points where residual $|\mathcal{R}_\theta(x)|$ is high.
2. **Adaptive Weighting**
   - **Solves** Gradient imbalance across loss terms.
   - Rescales weights so IC/BCs aren't dominated by PDE.
3. **Causality Training**
   - **Solves** Temporal instability & unphysical states.
   - Enforces sequential learning from $t = 0 \to T$.
4. **Neural Operators**
   - **Solves:** Re-training cost for new ICs/parameters.
   - Learns operator mappings ($f \mapsto u$) via FNO/DeepONet, etc.

Advanced Techniques for PINN Convergence

1. **Adaptive Sampling (R3, RAR)**
   - **Solves:** Localized shocks & steep gradients.
   - Concentrates points where residual $|\mathcal{R}_\theta(x)|$ is high.
2. **Adaptive Weighting**
   - **Solves** Gradient imbalance across loss terms.
   - Rescales weights so IC/BCs aren't dominated by PDE.
3. **Causality Training**
   - **Solves** Temporal instability & unphysical states.
   - Enforces sequential learning from $t = 0 \to T$.
4. **Neural Operators**
   - **Solves:** Re-training cost for new ICs/parameters.
   - Learns operator mappings ($f \mapsto u$) via FNO/DeepONet, etc.

PDE Variational Formulations (Deep Ritz/Galerkin) Solve: High-order derivative cost & non-smooth solutions via weak/energy forms.

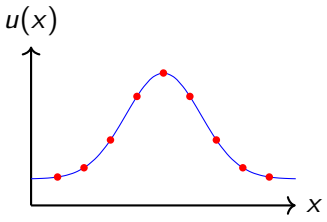
I. Adaptive Sampling in PINNs

Motivation:

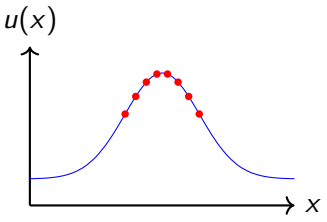
- Uniform sampling may waste resources in smooth regions.
- Some regions (e.g., sharp gradients, boundary layers) require denser sampling.

Idea: Dynamically adjust the sampling distribution based on PDE residuals.

Uniform Sampling



Adaptive Sampling



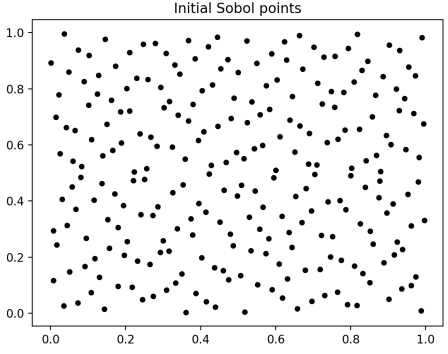
Benefits:

- Faster convergence and improved accuracy.
- Better resolution in critical regions.

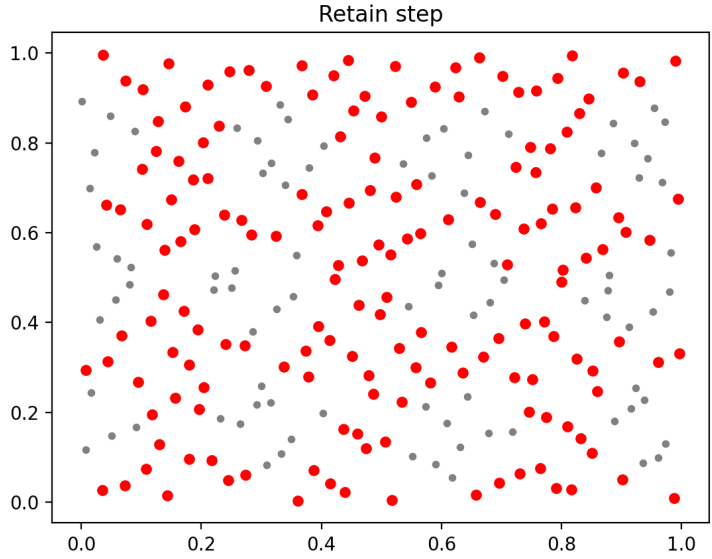
R3 Sampling: Idea

Retain–Resample–Release (R3), Daw et al.'22

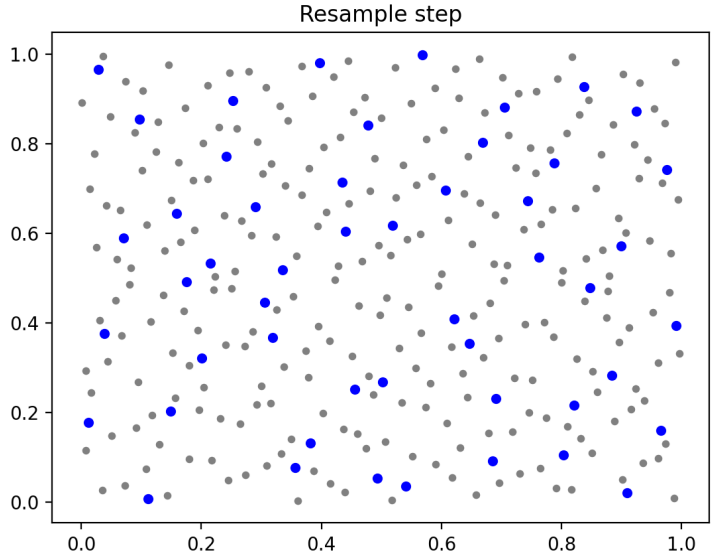
- **Retain**: Keep points with high residuals (where $\mathcal{L}_{\text{PDE}}$ is "large")
- **Resample**: Add new points across the domain to explore
- **Release**: Discard points with consistently low residuals



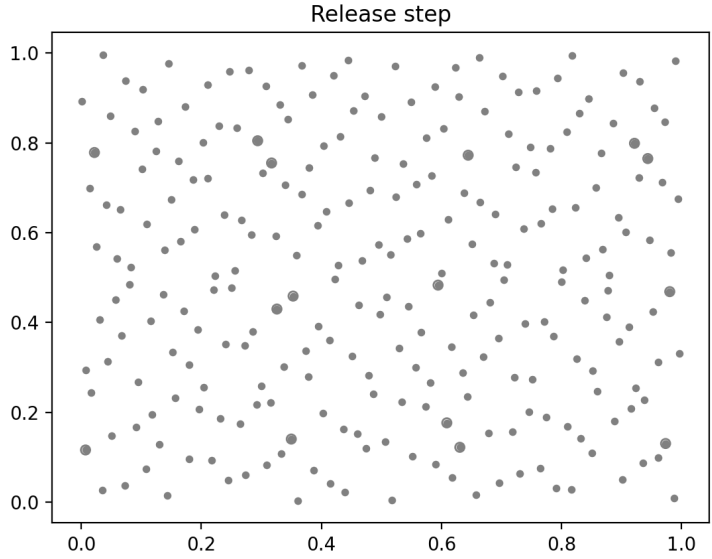
Initial Sobol collocation points



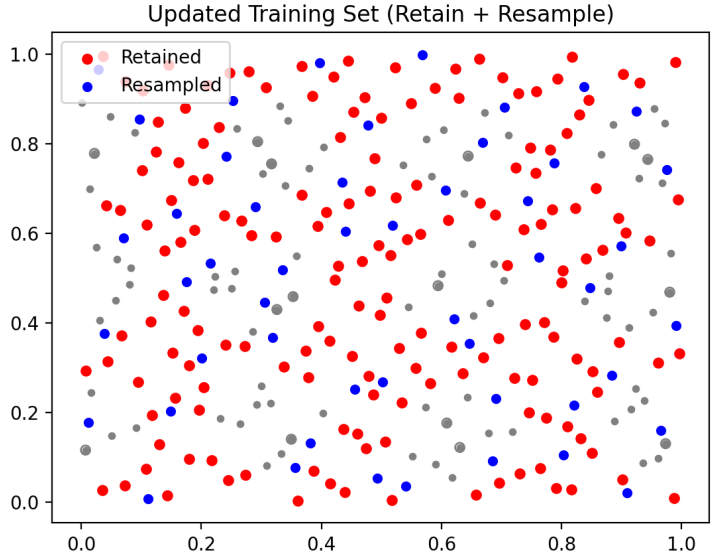
High-residual points are **retained**.



New Sobol points are **resampled**.



Low-residual points are **released**.



Final collocation set after one cycle: **retained** + **resampled** points.

R3 example

Training Loop with R3

```
points = Sobol.sample(N)
for epoch in range(n_epochs):
    # Compute residuals
    residuals = PDE_residual(model, points)
    # Retain "hard" points
    retain = points[residuals > tau_high]
    # Resample new points
    resample = Sobol.sample(M)
    # Release low-residual points
    release = points[residuals < tau_low]
    # Update training set
    points = retain + resample
```

- **Benefit:** Focuses capacity dynamically on steep gradients, moving fronts, and sharp interfaces.

The Optimization Pathology:

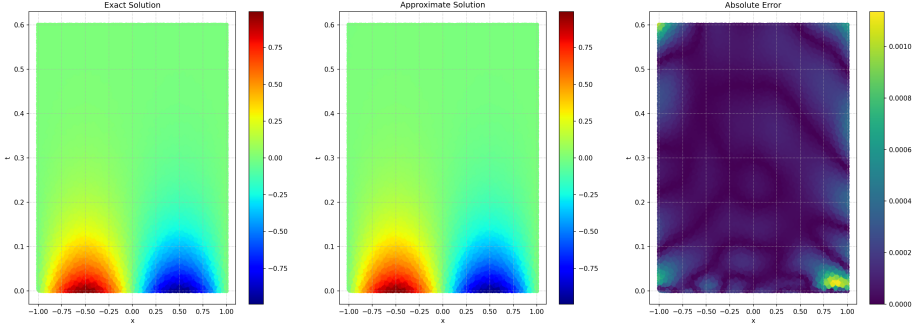
- Multi-objective loss minimization:

$$\mathcal{L} = \lambda_{\text{IC}}\mathcal{L}_{\text{IC}} + \lambda_{\text{BC}}\mathcal{L}_{\text{BC}} + \lambda_{\text{PDE}}\mathcal{L}_{\text{PDE}}$$

- Loss gradients differ by orders of magnitude, causing standard optimizer to favor one term.

Remark: Core Challenge

The main difficulty is **"gluing"** the interior PDE solution to the boundaries ($\partial\Omega$) and initial state ($t = 0$).

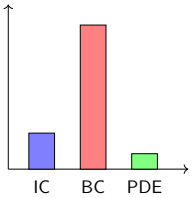


Dynamic Weight Normalization*

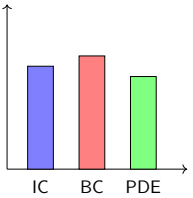
- Automatically update weights $\lambda_i^{(k)}$ during SGD/Adam iterations to balance gradient magnitudes across all objectives:

$$\lambda_i^{(k)} \propto \frac{1}{\|\nabla_\theta \mathcal{L}_i\|} \quad \Longrightarrow \quad \lambda_{\text{IC}}\|\nabla_\theta \mathcal{L}_{\text{IC}}\| \approx \lambda_{\text{BC}}\|\nabla_\theta \mathcal{L}_{\text{BC}}\| \approx \lambda_{\text{PDE}}\|\nabla_\theta \mathcal{L}_{\text{PDE}}\|$$

Unweighted Gradients (Imbalanced)



Balanced Gradients (Adaptive)



*Xiang et al. '22; **Self-adaptive loss balanced PINNs for the incompressible NS equations**.

Causality Violation in PINNs:

- Standard PINNs optimize all $t \in [0, T]$ **simultaneously**.
- Trains future states ($t_2 > t_1$) before resolving past (t_1).
- Often leads to unphysical steady states.

Causality Principle (Wang et al. '22)[a]

- Sequential learning from $t = 0 \to T$.
- Time t_k is trained *only* when past residuals ($t < t_k$) are minimized.

[a] **Respecting causality is all you need for PINNs**.

Causality-Weighted Loss

Partition $[0, T]$ into M slices $0 = t_1 < t_2 < \cdots < t_M = T$:

$$\mathcal{L}_{\text{PDE}}(\theta) = \sum_{k=1}^{M} w_k \mathcal{L}_k(\theta), \quad w_k = \exp\left(-\epsilon \sum_{j=1}^{k-1} \mathcal{L}_j(\theta)\right)$$

$\mathcal{L}_{<k} \gg 0 \implies w_k \approx 0$ (future suppressed) | $\mathcal{L}_{<k} \to 0 \implies w_k \to 1$ (future unlocked)

Example: 1D Heat Equation

Model Problem: $\partial_t u = u_{xx}$ on $(0, T] \times (-1, 1)$, with $u(0, x) = h(x)$.

Standard Loss (Non-Causal)

$$\mathcal{L}_{\text{PDE}} = \frac{1}{N} \sum_{i=1}^{N} |\partial_t u_\theta(t_i, x_i) - \partial_{xx} u_\theta(t_i, x_i)|^2$$

- All points (t_i, x_i) have weight $1/N$.
- **Risk:** Fits $t \approx T$ while failing on transient state at $t \approx 0$.

Causal Loss

$$\mathcal{L}_{\text{PDE}}(\theta) = \sum_{k=1}^{M} w_k \cdot \underbrace{\frac{1}{N_k} \sum_{i \in \mathcal{I}_k} |\partial_t u_\theta - \partial_{xx} u_\theta|^2}_{\mathcal{L}_k \text{ (residual on time slice } t_k)}$$

- $w_1 = 1$ (anchors initial state).
- $w_k = \exp\left(-\epsilon \sum_{j<k} \mathcal{L}_j\right)$ delays learning at t_k until $t_{<k}$ converges.

Takeaway: Guarantees sequential propagation ($t = 0 \to T$), preventing failure on transient and stiff dynamics.

IV. Operator Learning: Motivation & Concept

Standard PINN Setting

- **Instance-Specific:** Learns a single solution $u_\theta(t, x)$ for one fixed set of IC/BCs or parameters.
- **High Cost:** Any change in forcing $f(x)$ or BC, $u_0(x)$, PDE coeff. requires retraining from scratch.

The Operator Paradigm

Approximates an **infinite-dimensional mapping**:

$$\mathcal{G}_\theta : \mathcal{A} \to \mathcal{U}, \quad f \mapsto u$$

where f represents input functions (source terms, IC/BCs, or parameter fields).

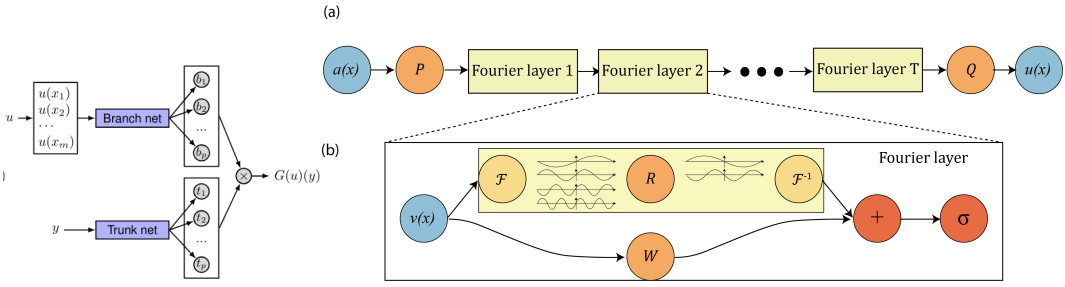
Key Properties

- **Fast Inference:** After offline training, evaluating $\mathcal{G}_\theta(f)$ for any input f requires only a single forward pass.
- **Resolution Invariance:** Evaluates solutions across varying grid resolutions (*architecture-dependent*, e.g., FNO/DeepONet vs. CNNs).

Neural Architectures for Operator Learning

Popular ones (Non-Exhaustive): Lu et al., Li et al., Ronneberger et al., Geiger et al.

- **DeepONet:** Branch–Trunk architecture encoding input and query locations.
- **Fourier Neural Operators (FNO):** Spectral parameterization via FFT for *resolution independence*.
- **U-Net:** Multi-scale convolutional operators for structured domain representations.
- **Geometric & Symmetry-Aware:** Equivariant/Graph Neural Operators, etc.

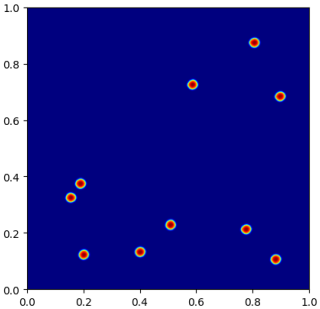


Left: DeepONet | Right: FNO (Spectral Layer)

Applications: Phase-Field Dendritic Growth

Goal: Operator learning for coupled phase-field solidification (H. et al., '25).

- **Coupled System:** Solves phase (φ) and temperature (U) with anisotropic surface energy.
- **Generalization:** Trains on 1–3 seeds $\rightarrow$ predicts complex multi-grain structures.
- **Speedup:** $3\times$–$4\times$ faster than high-order spectral solvers (SAV).

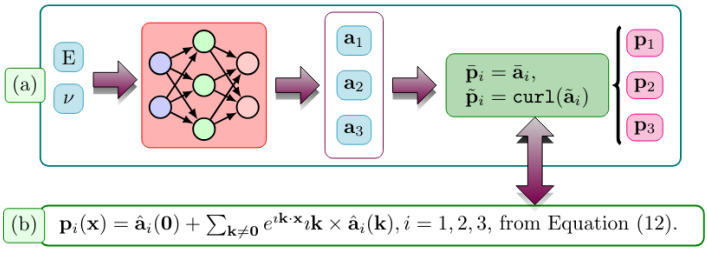


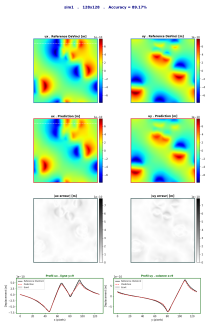
Neural Operator Surrogate: Multi-grain phase-field evolution

Applications: Stress Field Surrogates

Goal: Replace costly spectral solvers for mechanical equilibrium ($\nabla \cdot \boldsymbol{\sigma} = 0$) using FNOs (Khorrami et al. '24, Oubaha et al. '26).

- **Task:** Inputs (geometry, properties, constraints) $\rightarrow$ Displacement fields (u_x, u_y).
- **FNO Approaches:** Pure data-driven baseline, physics-informed ($\mathcal{L}_{\text{equil}}$), and iterative equilibrium solver.





Objective: Recover unknown parameter fields (e.g., diffusivity $\kappa(x)$) from sparse, noisy observations u_{obs}.

Forward vs. Inverse Formulations

- **Forward PDE:** $\partial_t u = \kappa(x)\Delta u$ with known $\kappa(x)$.
- **Physics-Informed Loss:**

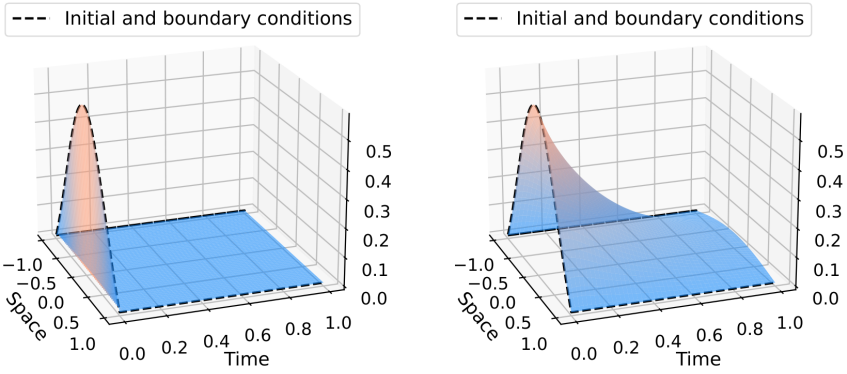
$$\mathcal{L}(\theta, \kappa) = \lambda_{\text{data}}\mathcal{L}_{\text{data}} + \lambda_{\text{PDE}}\mathcal{L}_{\text{PDE}}$$

Operator Advantage

- Standard inverse solvers re-run costly forward PDEs in an optimization loop.
- **Operator Inversion:** Neural operators learn direct mappings $u_{\text{obs}} \mapsto \kappa(x)$, enabling real-time parameter identification.

PINNs Can "Overfit"

PINNs can cheat and produce unphysical solutions (e.g., 1D heat equation).



Inconsistent PINN (left) vs. Exact Solution (right) (Doumèche et al., '23)

Fix: Add L^2-regularization on parameters θ:

$$\mathcal{L}_{\text{reg}} = \mathcal{L}_{\text{phys}} + \lambda_\theta \|\theta\|_2^2$$

Takeaways & Summary

- **Standard PINNs:** Unsupervised PDE solvers via automatic differentiation, but prone to stiff optimization pathologies (gradient imbalances, boundary mismatch), lack of theoretical guarantees (convergence, error bounds)
- **Advanced Techniques & Generalization:**
  - **Residual-based Adaptive sampling:** $x_{\text{new}} \sim P(x) \propto |\mathcal{R}_\theta(x)|^p$.
  - **Adaptive Weighting:** $\lambda_{\text{IC}}\|\nabla_\theta \mathcal{L}_{\text{IC}}\| \approx \lambda_{\text{BC}}\|\nabla_\theta \mathcal{L}_{\text{BC}}\| \approx \lambda_{\text{PDE}}\|\nabla_\theta \mathcal{L}_{\text{PDE}}\|$.
  - **Causality Training:** $w_i = \exp\left(-\epsilon \sum_{j<i} \mathcal{L}_{\text{PDE}}(t_j)\right)$.
  - **Operator Learning:** Extends PINNs to function-space mappings ($\mathcal{G}_\theta : f \mapsto u$) for instant zero-shot evaluation across inputs.

Now, it's time to **get your hands dirty!**

Outline

What is JAX?

- A high-performance numerical computing library for Python.
- Combines NumPy-like API with automatic differentiation.
- Supports GPU/TPU acceleration seamlessly.

Key Features

- `grad`: Automatic differentiation.
- `jit`: Just-In-Time compilation for speed.
- `vmap`: Vectorization across batch dimensions.
- `pmap`: Parallelism across multiple devices.

Hands-on Session (2h)

- **Notebook 1 — JAX Basics:** Core concepts and functional setup.
- **Notebook 2 — Intro to PINNs:** Solving the 1D Heat Equation.
- **Notebook 3 — Advanced Methods:** Adaptive sampling (R3) and causality on 2D Allen-Cahn equation; First Steps with FNO.

Allen-Cahn Equation: A Crash Course

Phase-Field $u(t, \mathbf{x}) \in [-1, 1]$ eliminates boundary tracking by describing microstructures continuously:

- $u = +1$**:** Phase A (e.g., Grain A/Solid)
- $u = -1$**:** Phase B (e.g., Grain B/Liquide)
- $u \in (-1, 1)$**:** Diffuse interface transition zone of width $\mathcal{O}(\varepsilon)$.

Ginzburg-Landau Free Energy:

$$E(u) = \int_\Omega \underbrace{\frac{1}{2}|\nabla u|^2}_{\text{Interfacial Penalty}} + \underbrace{\frac{W(u)}{\varepsilon^2}}_{\text{Bulk Free Energy}} \, \mathrm{d}\mathbf{x}, \quad W(u) = \frac{1}{4}(1 - u^2)^2$$

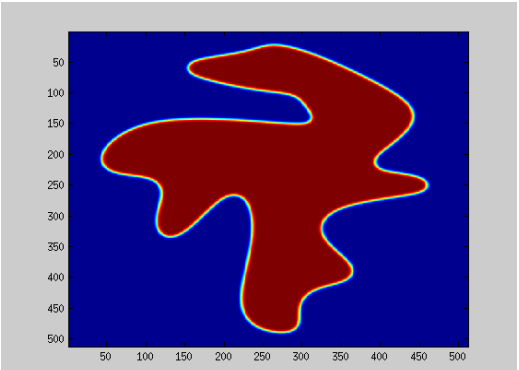
- **Double-well** $W(u)$**:** Drives phase separation toward two global minima $u = \pm 1$.
- **Gradient** $|\nabla u|^2$**:** Penalizes sharp interfaces, smoothing the transition.

Allen-Cahn Equation: Dynamics & Coarsening

Microstructure minimizes total energy over time: $\partial_t u = -\varepsilon^2 \nabla_{L^2} E(u)$

$$\partial_t u = \varepsilon^2 \Delta u - W'(u) = \varepsilon^2 \Delta u - (u^3 - u)$$

- **Diffusion ($\varepsilon^2 \Delta u$):** Spreads out interfacial energy.
- **Reaction ($-(u^3 - u)$):** Forces bulk points back to equilibrium states $u = \pm 1$.
- **Interface velocity** $v = -\kappa$ (curvature) $\implies$ small/curved grains shrink, driving domain coarsening and grain growth.



References

- Raissi et al. *Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations*, Journal of Computational Physics, 2019.
- Cuomo et al. Scientific Machine Learning through Physics-Informed Neural Networks: Where we are and What's next, 2022.
- Wu et al. *A comprehensive study of non-adaptive and residual-based adaptive sampling for physics-informed neural networks.* Computer Methods in Applied Mechanics and Engineering, vol. 404, p. 115671, 2023.
- Li et al. *Physics-informed neural operator for learning partial differential equations.* ACM/IMS Journal of Data Science, 1(3), 1-27, 2024.
- Fidle CNRS : Online introduction courses to Deep Learning (French), PINNS, 2022–2026.

Thank you for your attention!

... any questions ?